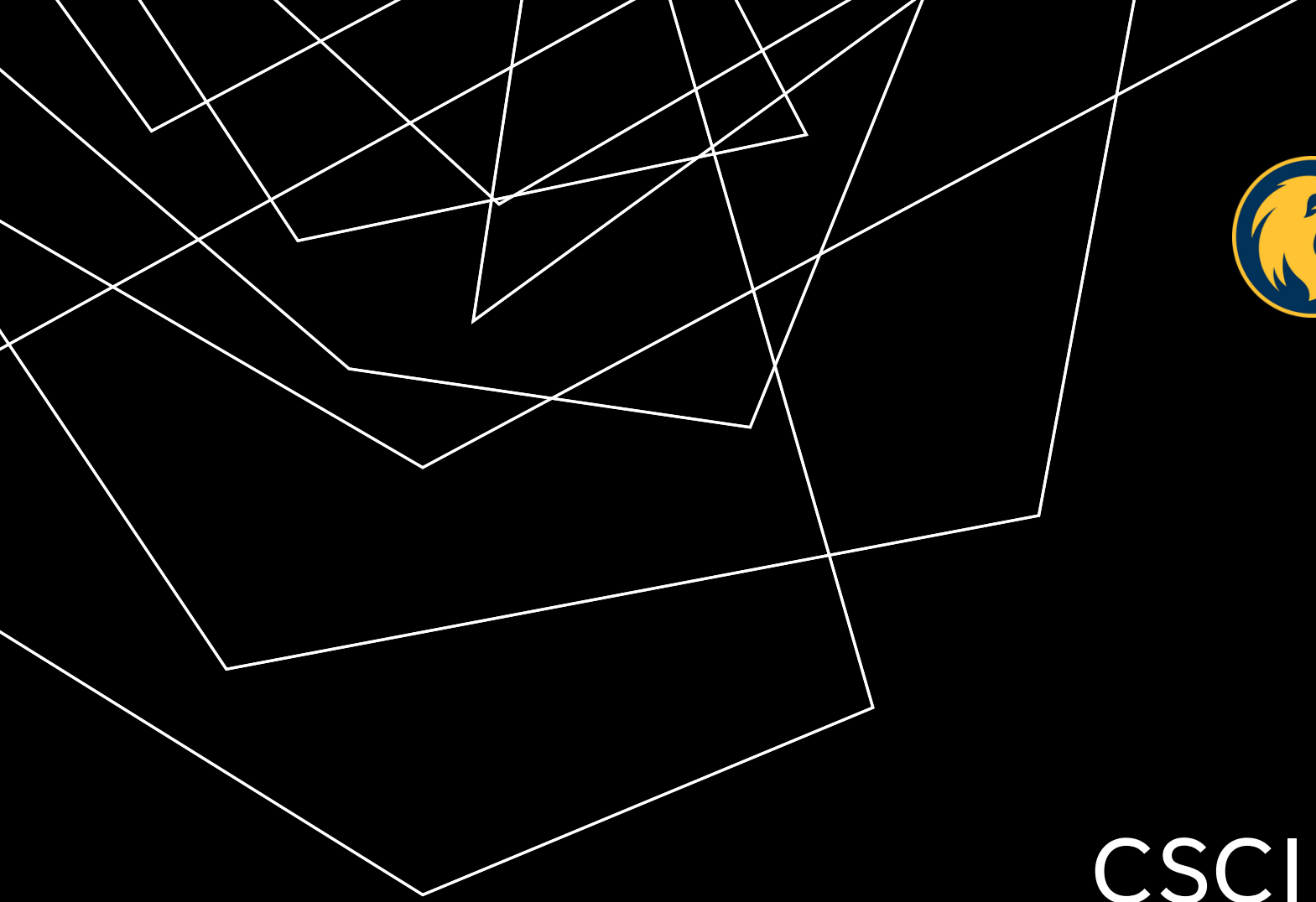
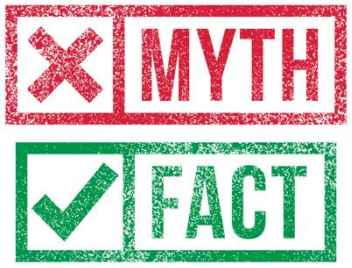


EAST TEXAS A&M
UNIVERSITY

CSCI 352.61E
DIGITAL FORENSICS

INVESTIGATE DATA LEAKAGE CASE

Keywords: Update Sequence Number (USN) Journaling



➡ If I delete a file from 

and change the file entry, no one will know which file I have deleted.

OVERVIEW

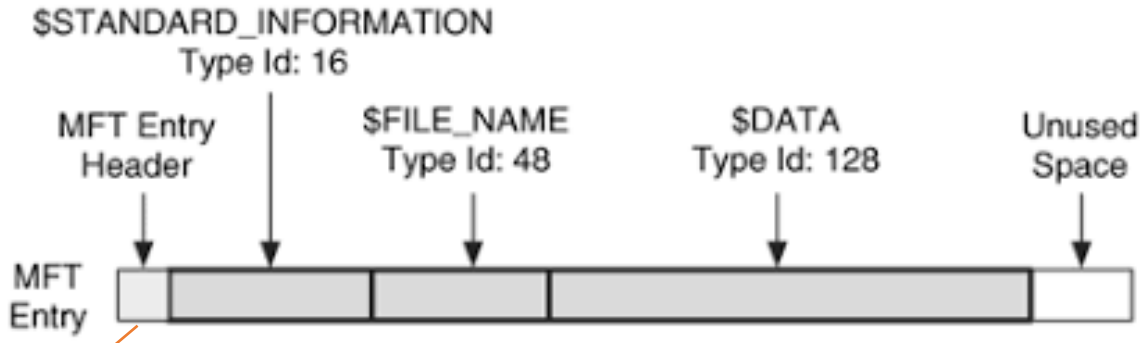
- $MFT: Master File Table
- USN Journal
- Identify all traces related to changing of files in Windows Desktop

$MFT: MASTER FILE TABLE

$MFT HAS MANY ENTRIES (RECORDS)

- MFT starts as small as possible and expands when more entries are needed.
- $MFT is file, $MFT has an entry for itself.

Entry (1k)
Entry (1k)
Entry (1k)
Entry (1k)
Entry (1k)
Entry (1k)



Reference #	
Entry/Record #	Seq#
0x0000 0000 1421	0x0002
0x0000 0000 1422	0x0005
0x0000 0000 1423	0x000a
0x0000 0000 1424	0x0001

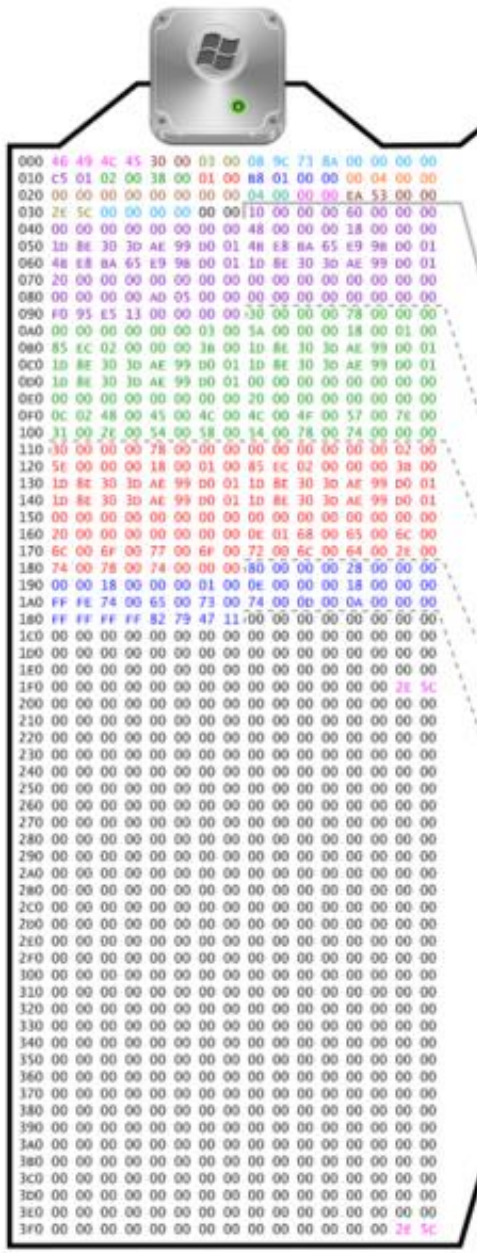
$MFT FILE REFERENCE NUMBER

- MFT File Reference # =MFT sequence # + MFT entry #
- MFT entry #
  - Each represents one file
  - ID starts from zero,
  - zero being the MFT file itself.
- MFT sequence #
  - # of times that one existing MFT entry have been used
  - e.g., a file is deleted and its MFT entry to be reused for a new file.
    - its sequence number would be incremented from 1
  - benefits: determine a crashed/corrupted state
    - check different MFT sequence #

MASTER FILE TABLE RECORD

Invoke-IR

BY: JARED ATKINSON
TEMPLATE BY: ANGE ALBERTINI



FILE RECORD HEADER

FIELDS	VALUES
magic	FILE
offset to us	0x30
size of us	0x03
logical sequence number	8A739C08
sequence number	0x1C5
hardlinks	0x02
offset to attributes	0x38
flags	0x01
real size	0x1B8
allocated size	0x400
reference to base	0x0000000000000000
next attribute id	0x04
alignment bytes	0x00
record numbers	0x53EA
update sequence	0x02 0x00
update sequence array	0x00 0x00 0x00 0x00

ATTRIBUTES

- $STANDARD_INFORMATION ATTRIBUTE
- $FILE_NAME ATTRIBUTE
- $FILE_NAME ATTRIBUTE
- $DATA ATTRIBUTE

FLAGS

0X01 - IN USE
0X02 - DIRECTORY

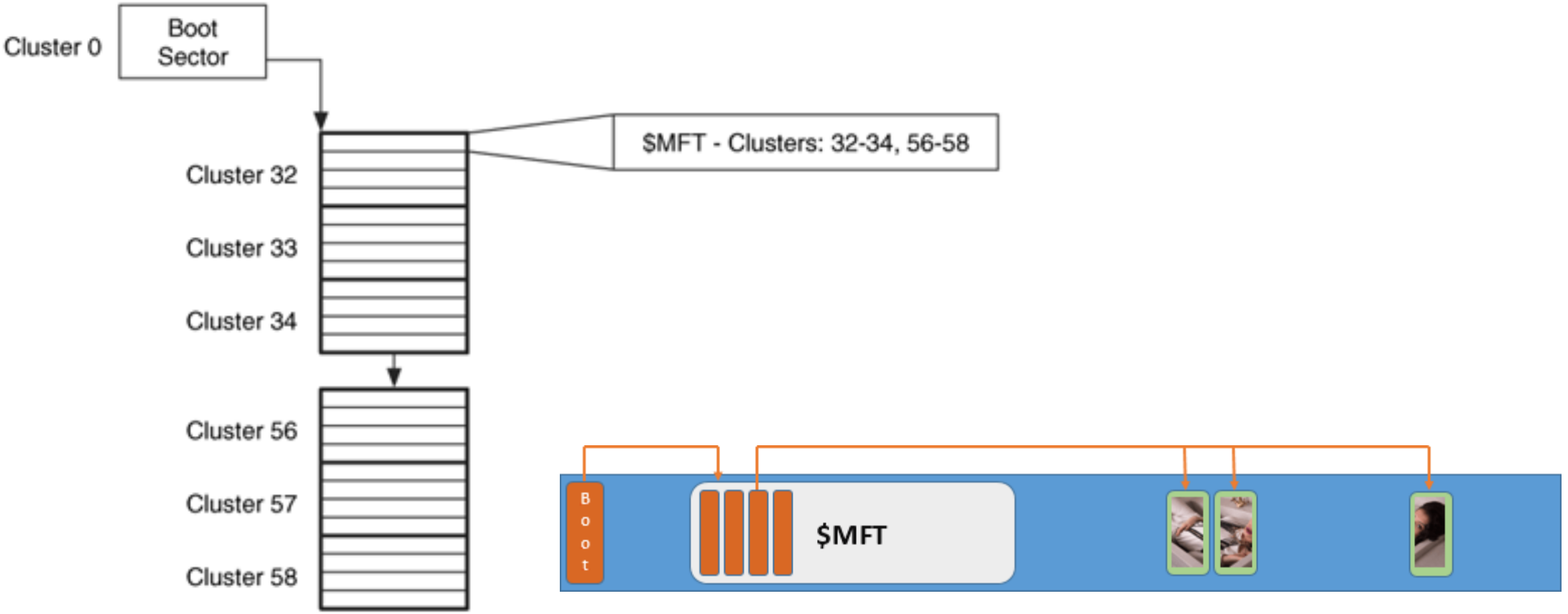
REAL VS ALLOCATED SIZE

Allocated Size - Size of allocated disk space. This size will be divisible by the size of a disk cluster.

Real Size - Actual size of file contents. This size is the one referenced by the "dir" command.

If real and allocated size are 0, then the file's contents are contained within a resident data attribute in the file's MFT record.

BOOT SECTOR AND $MFT

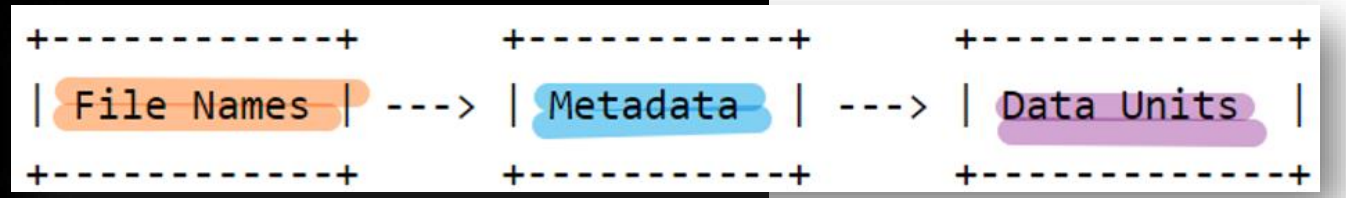


https://flylib.com/books/en/2.48.1/mft_concepts.html

$MFT FILE IS ONE OF SYSTEM METADATA FILES

```
/bin/bash 12
root@kali:~/lab#
root@kali:~/lab# fls -o 206848 cfreds_2015_data_leakage_pc.dd
d/d 273-144-6:  Program Files (x86)
d/d 486-144-5:  Users
r/r 4-128-4:    $AttrDef
r/r 8-128-2:    $BadClus
r/r 8-128-1:    $BadClus:$Bad
r/r 6-128-4:    $Bitmap
r/r 7-128-1:    $Boot
d/d 11-144-4:   $Extend
r/r 2-128-1:    $LogFile
r/r 0-128-1:    $MFT
r/r 1-128-1:    $MFTMirr
d/d 57-144-5:   $Recycle.Bin
r/r 9-128-8:    $Secure:$SDS
r/r 9-144-16:   $Secure:$SDH
r/r 9-144-17:   $Secure:$SII
r/r 10-128-1:   $UpCase
r/r 3-128-3:    $Volume
```

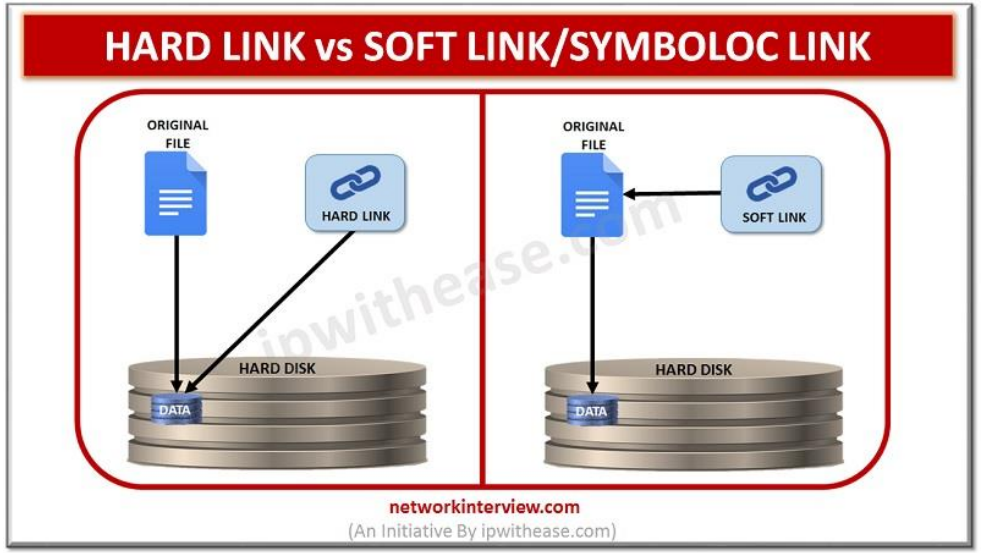
Contains the information about all files and directories



A MFT ENTRY EXAMPLE: *GMREADME.TXT*

```
┌──(root💀kali80)-[/home/student/lab]
└─# fls -r -o 206848 cfreds_2015_data_leakage_pc.dd | grep "gmreadme.txt"

+++ r/r 27331-128-3:     gmreadme.txt
+++ r/r 27330-128-3:     gmreadme.txt
+++ r/r 27331-128-3:     gmreadme.txt
+++ r/r 27330-128-3:     gmreadme.txt
```

four copies in different folders (different parents). Two point to the same inode (hard links)



https://flatcap.github.io/linux-ntfs/ntfs/concepts/file_record.html

changed every time the record is modified.

$STD timestamp
- can be modified by user/application-level processes.
- Easy to be changed for anti-forensics
- collected by Windows explorer, fls, mactime, timestamp, find

$FILE_NAME timestamps
- can only be changed by the system kernel.
- Difficult to be modified by users/application.
- More important
- *only create/copy/volume move change*

Anti-forensics indication:
$FILE_NAME time occurs after the $STANDARD_INFORMATION Creation Time.

```
┌──(root💀kali80)-[/home/student/lab]
└─# istat -o 206848 cfreds_2015_data_leakage_pc.dd 27331

MFT Entry Header Values:
Entry: 27331        Sequence: 1
$LogFile Sequence Number: 71967542
Allocated File
Links: 2

$STANDARD_INFORMATION Attribute Values:
Flags: Archive
Owner ID: 0
Security ID: 455  (S-1-5-80-956008885-3418522649-1831038044-1853292631-2271478464)
Created:        2009-07-13 18:13:39.513319900 (EDT)
File Modified:  2009-06-10 16:30:50.554964500 (EDT)
MFT Modified:   2015-03-25 07:11:30.834855500 (EDT)
Accessed:       2009-07-13 18:13:39.513319900 (EDT)

$FILE_NAME Attribute Values:
Flags: Archive
Name: gmreadme.txt
Parent MFT Entry: 2393  Sequence: 1
Allocated Size: 648     Actual Size: 646
Created:        2015-03-25 07:11:29.696053500 (EDT)
File Modified:  2015-03-25 07:11:29.696053500 (EDT)
MFT Modified:   2015-03-25 07:11:29.696053500 (EDT)
Accessed:       2015-03-25 07:11:29.696053500 (EDT)

$FILE_NAME Attribute Values:
Flags: Archive
Name: gmreadme.txt
Parent MFT Entry: 4273  Sequence: 1
Allocated Size: 0       Actual Size: 0
Created:        2015-03-25 07:11:29.696053500 (EDT)
File Modified:  2015-03-25 07:11:29.696053500 (EDT)
MFT Modified:   2015-03-25 07:11:29.696053500 (EDT)
Accessed:       2015-03-25 07:11:29.696053500 (EDT)

Attributes:
Type: $STANDARD_INFORMATION (16-0)   Name: N/A   Resident   size: 72
Type: $FILE_NAME (48-4)   Name: N/A   Resident   size: 90
Type: $FILE_NAME (48-2)   Name: N/A   Resident   size: 90
Type: $DATA (128-3)   Name: N/A   Non-Resident   size: 646  init_size: 646
1199027
```

MFT File Reference Number (8 bytes) =
MFT Entry (inode #, 6 bytes): 27331 +
MFT sequence (Number of times this mft record has been reused 2 bytes): 1

B/created/birth: when a file a birth/created/Copied/Moved
File M/mtime: the last time the content $DATA a file has been changed
MFT M: the file's metadata changes (C)
A/atime: access time of $DATA

- Same files, two hard links
- Long file name has two link
  - DOS-compatible short name (EXTRE~1.TXT).

MACB TIMESTAMPS FOR DIFFERENT FILE SYSTEMS (FROM SLEUTHKIT)

File System	M	A	C	B
Ext4	Modified	Accessed	Changed	Created
Ext2/3	Modified	Accessed	Changed	N/A
FAT	Written	Accessed	N/A	Created
NTFS	$Data Modified	Accessed	$MFT Modified	Created
UFS	Modified	Accessed	Changed	N/A

Note: Difference version many produce different BMAC timestamp changes

General rules, but not always true because file operations may involve both app and system level APIs

Condition	Modified (is content modified?)	$MFT Modified?	Accessed (need to read file?)	Created (birth of a new file?)
When renaming a file	No	?	No	No
When moving a file between folders	No	?	No	No
When moving a file between disks or partitions	No	?	Time of move	No
When copying a file	No	?	Time of copy	Time of copy
When writing a new file	Time of creation	?	Time of creation	Time of creation
When modifying an existing file	Time modified	?	No	No
When accessing an existing file	No	?	Updated within an hour if enabled, otherwise Preserved	No

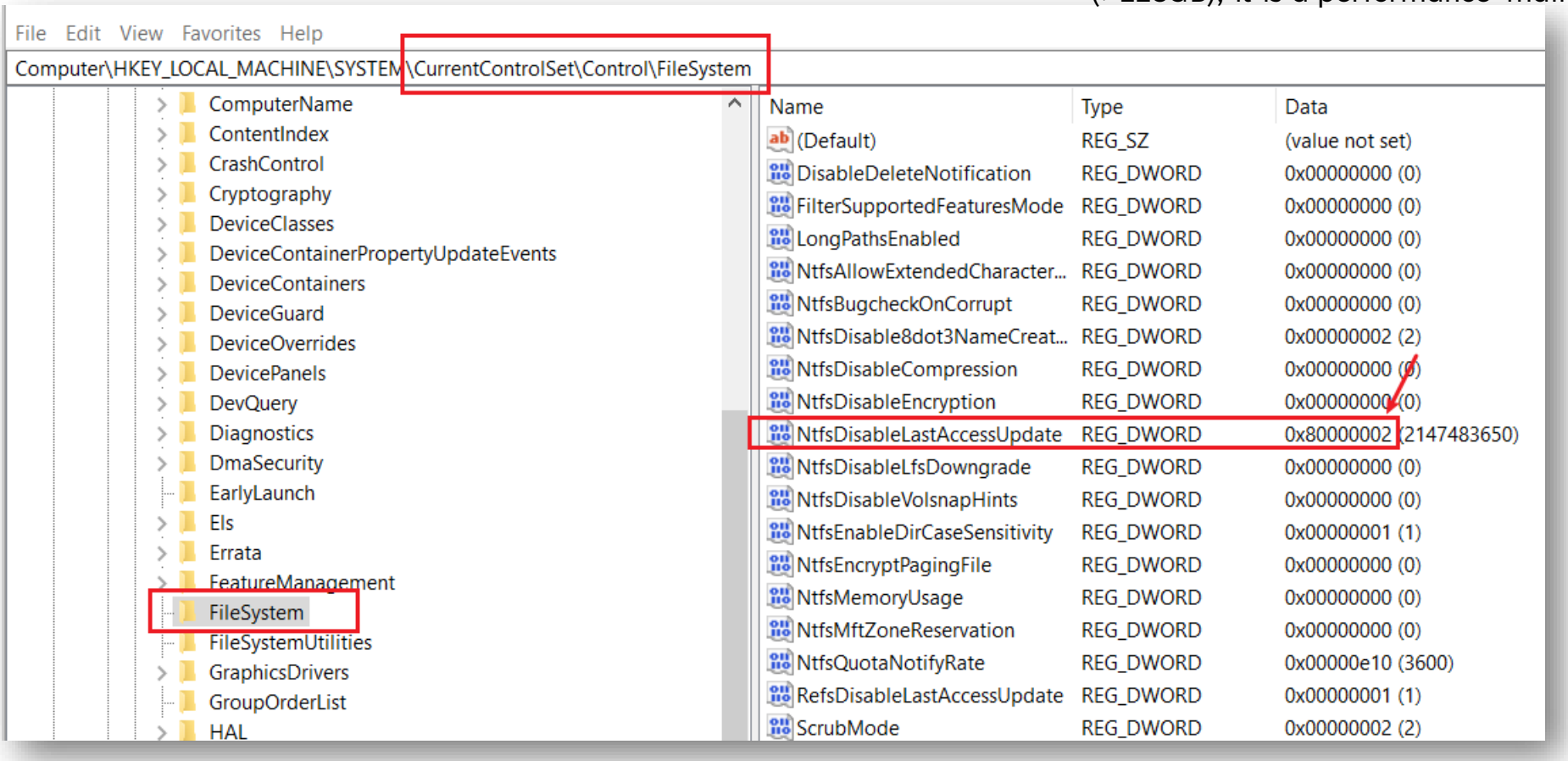
On various operating systems the update of the access time can be disabled. This means when a file is accessed the *atime* in the corresponding file system entry is not updated.

HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\FileSystem\NtfsDisableLastAccessUpdate

0 = User Managed, Last Access Updates Enabled
1 = User Managed, Last Access Updates Disabled
2 = System Managed, Last Access Updates Enabled (default)
3 = System Managed, Last Access Updates Disabled

'User Managed' mode
- disabled: the last access time stamp remains unchanged during the boot, regardless of the volume size.
- enabled: the OS changes it only when a user activity occur.

'System Managed' mode
- disabled: completely disables the Last Access Time Stamp feature for NTFS
- enabled: the last access time stamp is modified during the boot, once the OS mounts volumes. Windows 10 doesn't change it for large volumes (>128GB), it is a performance-maintenance trade off.



Windows® Time Rules[1]

$STANDARD_INFORMATION

File Creation	File Access	File Modification	File Rename	File Copy	Local File Move	Volume File Move (move via CLI)	Volume File Move (cut/paste via Explorer)	File Deletion
Modified – Time of File Creation	Modified – No Change	Modified – Time of Data Modification	Modified – No Change	Modified – Inherited from Original	Modified – No Change	Modified – Inherited from Original	Modified – Inherited from Original	Modified – No Change
Access – Time of File Creation	Access – Time of Access (No Change on NTFS Volumes > 128 GB)	Access – Time of Data Modification	Access – No Change	Access – Time of File Copy	Access – No Change	Access – Time of File Move via CLI	Access – Time of Cut/Paste	Access – No Change
Metadata – Time of File Creation	Metadata – No Change	Metadata – Time of Data Modification	Metadata – Time of File Rename	Metadata – Time of File Copy	Metadata – Time of Local File Move	Metadata – Inherited from Original	Metadata – Inherited from Original	Metadata – No Change
Creation – Time of File Creation	Creation – No Change	Creation – No Change	Creation – No Change	Creation – Time of File Copy	Creation – No Change	Creation – Time of File Move via CLI	Creation – Inherited from Original	Creation – No Change

$FILENAME

File Creation	File Access	File Modification	File Rename	File Copy	Local File Move	Volume File Move (move via CLI)	Volume File Move (cut/paste via Explorer)	File Deletion
Modified – Time of File Creation	Modified – No Change	Modified – No Change	Modified – No Change	Modified – Time of File Copy	Modified – No Change	Modified – Time of Move via CLI	Modified – Time of Cut/Paste	Modified – No Change
Access – Time of File Creation	Access – No Change	Access – No Change	Access – No Change	Access – Time of File Copy	Access – No Change	Access – Time of Move via CLI	Access – Time of Cut/Paste	Access – No Change
Metadata – Time of File Creation	Metadata – No Change	Metadata – No Change	Metadata – No Change	Metadata – Time of File Copy	Metadata – No Change	Metadata – Time of Move via CLI	Metadata – Time of Cut/Paste	Metadata – No Change
Creation – Time of File Creation	Creation – No Change	Creation – No Change	Creation – No Change	Creation – Time of File Copy	Creation – No Change	Creation – Time of Move via CLI	Creation – Time of Cut/Paste	Creation – No Change

Windows Time Rules based off of testing on Windows 10 Release version 1903

using .docx for experiments

$STANDARD_INFO

File Rename	Local File Move	Volume File Move	File Copy	File Access	File Modify	File Creation	File Deletion
Modified - No Change	Modified - No Change	Modified - No Change	Modified - No Change	Modified - No Change	Modified - Changed	Modified - Changed	Modified - No Change
Access - No Change	Access - No Change	Access - Changed	Access - Changed	Access - No Change	Access - Changed	Access - Changed	Access - No Change
Creation- No Change	Creation- No Change	Creation- Changed	Creation- Changed	Creation- No Change	Creation- No Change	Creation- Changed	Creation- No Change
Metadata - Changed	Metadata - Changed	Metadata - No change	Metadata - No change	Metadata - No Change	Metadata - Changed	Metadata - Changed	Metadata - No Change

$FILE_NAME

File Rename	Local File Move	Volume File Move	File Copy	File Access	File Modify	File Creation	File Deletion
Modified - No Change	Modified - No Change	Modified - Changed	Modified - Changed	Modified - No Change	Modified - Changed	Modified - Changed	Modified - No Change
Access - No Change	Access - No Change	Access - Changed	Access - Changed	Access - No Change	Access - Changed	Access - Changed	Access - No Change
Creation- No Change	Creation- No Change	Creation- Changed	Creation- Changed	Creation- No Change	Creation- No Change	Creation- Changed	Creation- No Change
Metadata - No change	Metadata - No change	Metadata - Changed	Metadata - Changed	Metadata - No Change	Metadata - Changed	Metadata - Changed	Metadata - No Change

CYBERFORENSICATOR.COM

using .txt for experiments

File Rename

		My test	Cyberforensicator	SANS
	'A' file	'A' renamed to 'B' file		
$STD Modification date (M)	2019-04-20 20:31:06.840866	2019-04-20 20:31:06.840866	No change	No change
$STD Access date (A)	2019-04-20 20:31:06.840866	2019-04-20 20:31:06.840866	No change	No change
$STD Metadata date (C)	2019-04-20 20:31:06.840866	2019-04-20 20:45:42.916199	Changed	Changed
$STD Creation date (B)	2019-04-20 20:31:06.840866	2019-04-20 20:31:06.840866	No change	No change
$FN Modification date (M)	2019-04-20 20:31:06.840866	2019-04-20 20:31:06.840866	No change	No change
$FN Access date (A)	2019-04-20 20:31:06.840866	2019-04-20 20:31:06.840866	No change	No change
$FN Metadata Entry date (C)	2019-04-20 20:31:06.840866	2019-04-20 20:31:06.840866	No change	No change
$FN Creation date (B)	2019-04-20 20:31:06.840866	2019-04-20 20:31:06.840866	No change	No change

File Modification

		My test	Cyberforensicator	SANS
	'A' file	modified 'A' file		
$STD Modification date (M)	2019-04-20 20:31:04.825193	2019-04-20 20:45:40.900618	Changed	Changed
$STD Access date (A)	2019-04-20 20:31:04.825193	2019-04-20 20:45:42.009945	Changed	No change
$STD Metadata date (C)	2019-04-20 20:31:04.825193	2019-04-20 20:45:40.900618	Changed	Changed
$STD Creation date (B)	2019-04-20 20:31:04.825193	2019-04-20 20:31:04.825193	No change	No change
$FN Modification date (M)	2019-04-20 20:31:04.825193	2019-04-20 20:31:04.825193	Changed	No change
$FN Access date (A)	2019-04-20 20:31:04.825193	2019-04-20 20:31:04.825193	Changed	No change
$FN Metadata Entry date (C)	2019-04-20 20:31:04.825193	2019-04-20 20:31:04.825193	Changed	No change
$FN Creation date (B)	2019-04-20 20:31:04.825193	2019-04-20 20:31:04.825193	No change	No change

Check here for comprehensive study results

keep track of changes to boxes

USN JOURNAL

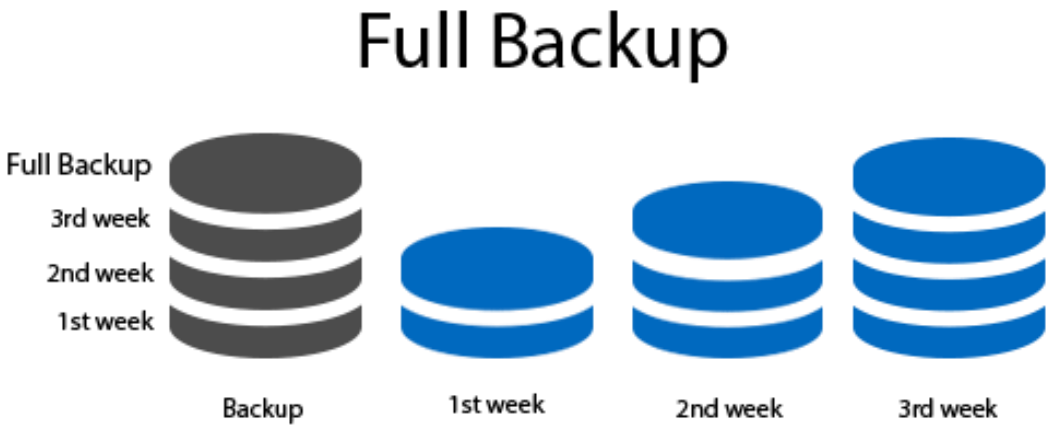
Update Sequence Number

Box ID	Box Name	Reason for changes	Change Date
1	My homework 1	Change answer 2	10/18/2022
2	My homework 2	Throw away	1/22/2023
3	My homework 1	Move to B5	10/2/2023

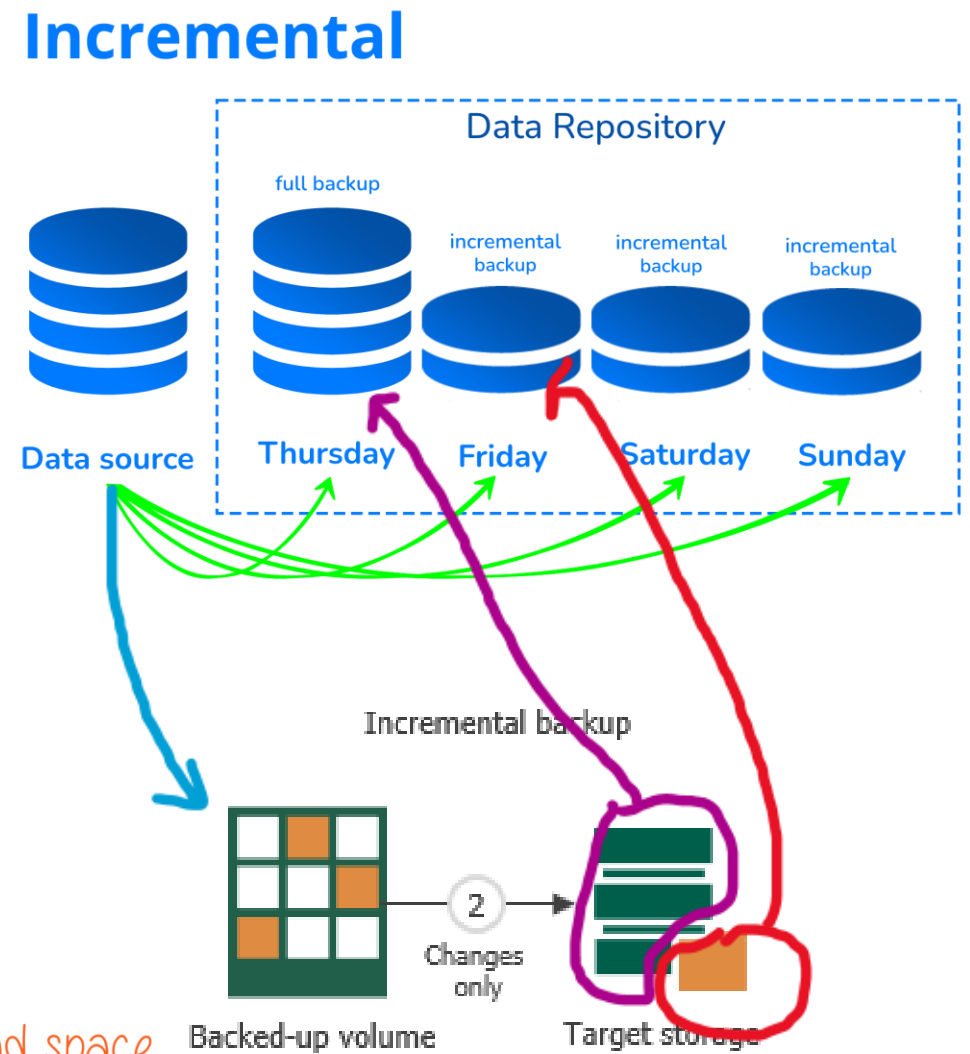
OUTLINES

- How to backup disks?
- What is USN Journal?
- The Structure of USN Journal
- What information is included in USN Journal (Journal Record)?
- How is the information used for digital forensics?

HOW TO BACKUP DISKS?



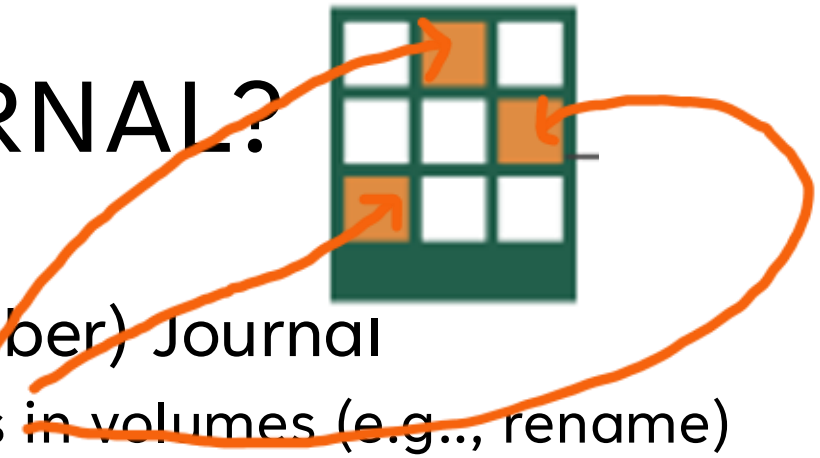
http://msbkffilerecovery.com/blog/wp-content/uploads/2019/01/1.png



data/uploads//2022/08/virt_incremental.png

Backup disks while saving time and space

WHAT IS USN JOURNAL?

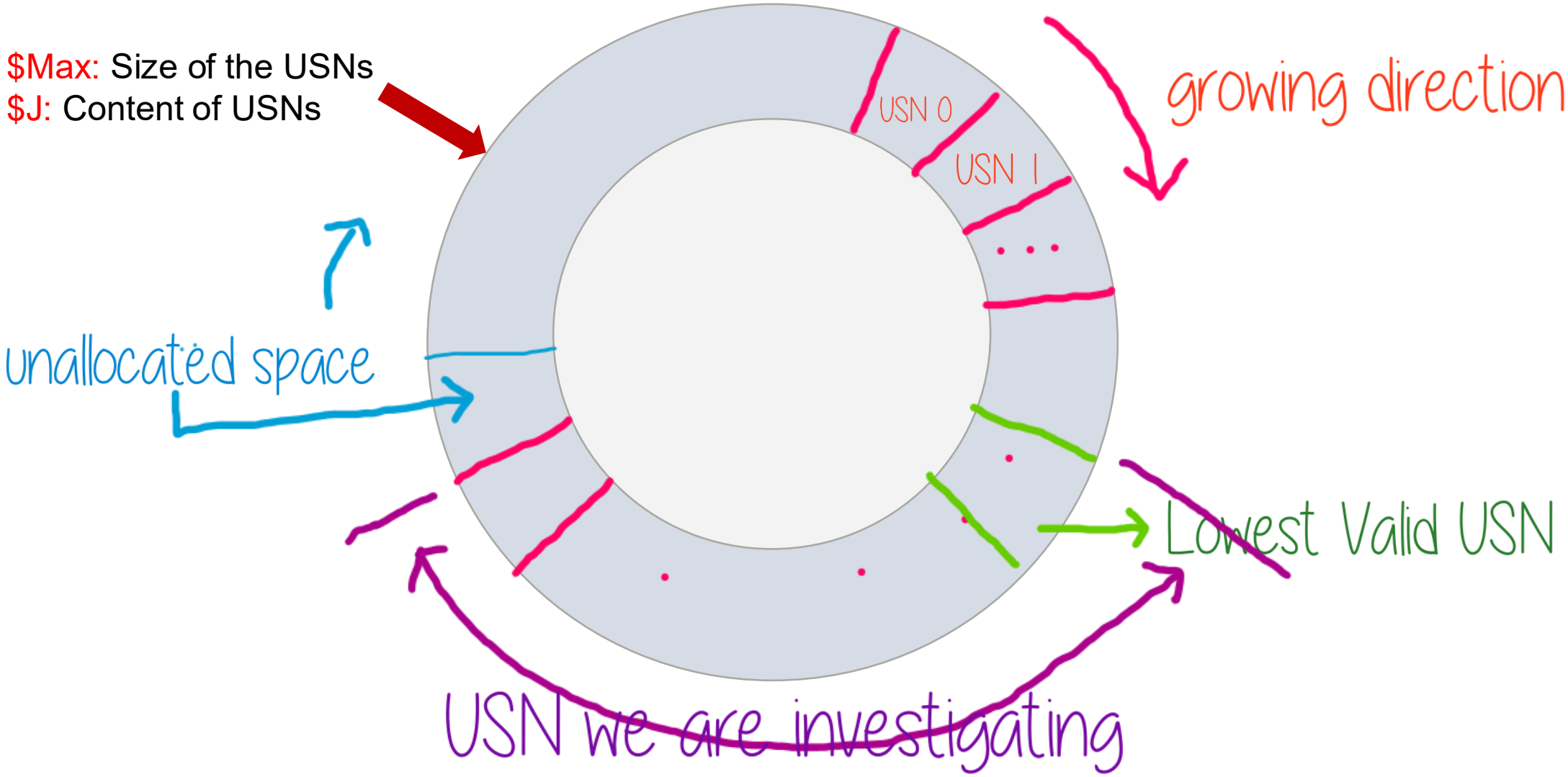


- USN (Update Sequence Number) Journal
  - keep tracking of changed files in volumes (e.g., rename)
  - Is a feature of the Windows NT file system (NTFS)
- Minimum Change information is saved
  - Doesn't need to include the details of data changes
    - When and what was done to the various objects (not HOW).
  - Changes are described using bit flags
- From Win7, Journal Function is activated by default
  - USN is a treasure trove of information for a forensic investigation.
  - Keep track of suspect's behaviors to volumes

INFORMATION OF USN JOURNAL RECORD

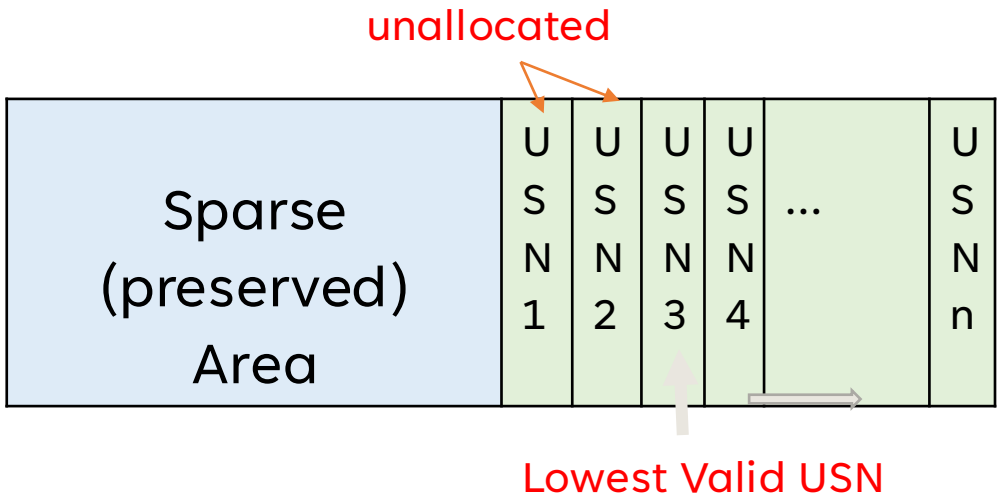
- USN of the record
- Time of change
- Reason for the change
- File/directory’s name
- File/directory’s attributes
- File/directory’s MFT reference number
- File reference number of the file’s parent directory
- Security ID to identifier of a user, user group
  - "S-1-5-21-3623811015-3361044348-30300820-1013"
- Information about the source of the change

THE STRUCTURE OF USN JOURNAL



THE DETAILS OF STRUCTURE OF USN JOURNAL

- "Sparse Area“ to preserve space efficiently: zero-filled
  - the file effectively does not consume any more disk space than needed
  - but can just seem to grow infinitely.
- The structure allows OS to keep the same size of the log data
- The new records are added at the end of the attribute.



GROW OF USN JOURNAL

- It begins as an empty file.
  - Whenever a change is made to the volume, a record is added to the file.
  - Each record is identified by a 64-bit Update Sequence Number (USN, for this reason Change Journals are sometimes called USN Journals).
  - Each record in the Change Journal contains the USN, the name of the file, and information about what the change was.
- It does not include all the data or details associated with the change.
  - Cannot be used to undo operations on files within NTFS
- Will be reused when journal reaches its maximum size
  - Clusters of the journal's disk space are marked unallocated by OS
  - Unallocated space can be extremely valuable

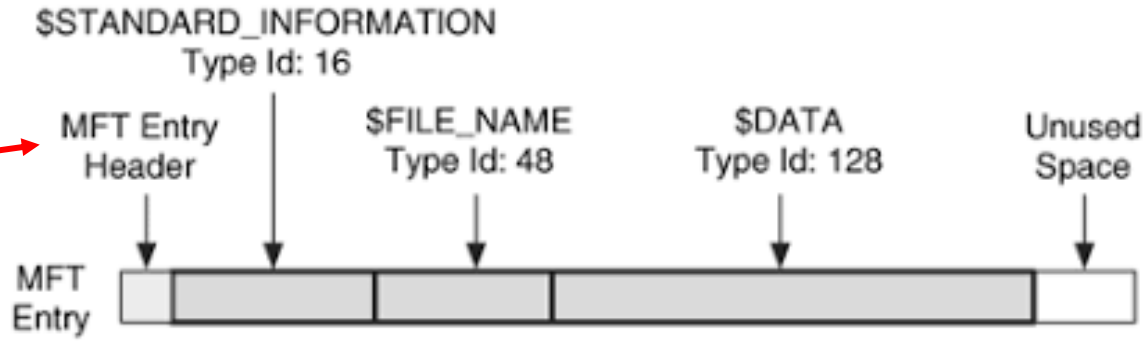
$EXTEND\$USNJRNL CONTAINS TWO ALTERNATE DATA STREAMS

Search for *$Extend\$UsnJrnl*

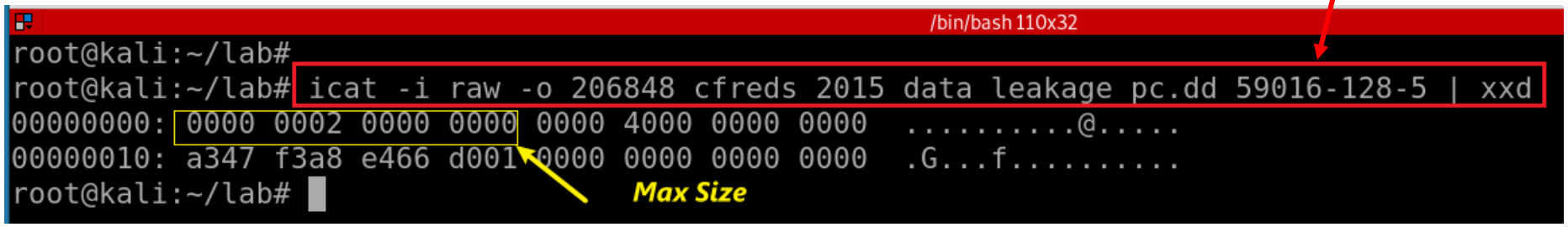
```
/bin/bash 68x26
root@kali:~/lab#
root@kali:~/lab# fls -rF -o 206848 cfreds_2015_data_leakage_pc.dd |
grep -P  "[\$]Extend/[\$]UsnJrnl" --color
r/r 59016-128-3:        $Extend/$UsnJrnl:$J
r/r 59016-128-5:        $Extend/$UsnJrnl:$Max
root@kali:~/lab#
```

[ ]: Match a single character

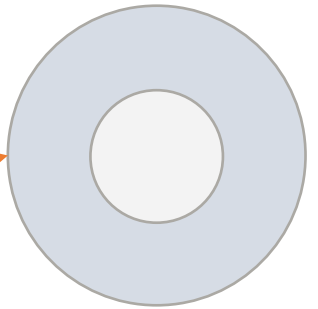
- $J: The actual change log records: the date and time of the change, the reason for the change, the Master File Table (MFT) entry, the parent entry and others.
- $Max: The meta data of change log: e.g., the maximum size.



$EXTEND\$USNJRNL: $MAX ATTRIBUTE



Offset	Size	Stored	Information Detail
0x00	8	Maximum Size	The maximum size of Journaling data
0x08	8	Allocation Size	The size of allocated area when new log data is saved.
0x10	8	Time	The creation time of "$*UsnJrnl*" file(FILETIME)
0x18	8	Lowest Valid USN	The least value of USN in current records With this value, investigator can approach the start point of first record within "$J" attribute



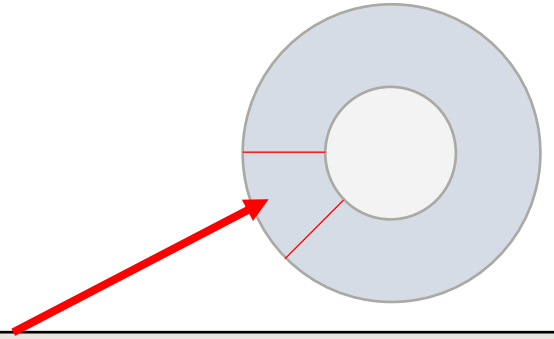
```
r/r 59016-128-3:        $Extend/$UsnJrnl:$J
r/r 59016-128-5:        $Extend/$UsnJrnl:$Max
```

VIEW *$EXTEND\$USNJRNL: $J $MFT ENTRY*

```
/bin/bash 110x32
root@kali:~/lab#
root@kali:~/lab# istat -i raw -o 206848 cfreds_2015_data_leakage_pc.dd 59016 | head -n 29
MFT Entry Header Values:
Entry: 59016        Sequence: 2
$LogFile Sequence Number: 386543835
Allocated File
Links: 1

$STANDARD_INFORMATION Attribute Values:
Flags: Hidden, System, Archive, Sparse
Owner ID: 0
Security ID: 257  (S-1-5-18)
Created:        2015-03-25 06:15:46.683689900 (EDT)
File Modified:  2015-03-25 06:15:46.683689900 (EDT)
MFT Modified:   2015-03-25 06:15:46.683689900 (EDT)
Accessed:       2015-03-25 06:15:46.683689900 (EDT)

$FILE_NAME Attribute Values:
Flags: Hidden, System, Archive
Name: $UsnJrnl
Parent MFT Entry: 11    Sequence: 11
Allocated Size: 0       Actual Size: 0
Created:        2015-03-25 06:15:46.683689900 (EDT)
File Modified:  2015-03-25 06:15:46.683689900 (EDT)
MFT Modified:   2015-03-25 06:15:46.683689900 (EDT)
Accessed:       2015-03-25 06:15:46.683689900 (EDT)

Attributes:
Type: $STANDARD_INFORMATION (16-0)   Name: N/A   Resident   size: 72
Type: $FILE_NAME (48-1)   Name: N/A   Resident   size: 82
Type: $DATA (128-3)   Name: $J   Non-Resident, Sparse   size: 69767168  init_size: 69767168
root@kali:~/lab#
```

MFT File Reference Number (8 bytes) =
MFT Entry (inode #, 6 bytes): 59016 +
MFT sequence (Number of times this mft record has been reused 2 bytes): 2

$J RECORD FORMAT



Offset	Size	Stored Information	Explanation
0x08	8	File Reference Number	The ordinal number of the file or directory for which this record notes changes.
0x10	8	Parent File Reference Number	The ordinal number of the directory where the file or directory that is associated with this record is located.
0x18	8	**Usn**	The USN of this record
0x20	8	**TimeStamp**	The standard UTC time stamp (FILETIME) of this record, in 64-bit format.
0x28	4	**Reason flag**	The flags that identify reasons for changes that have accumulated in this file or directory journal record since the file or directory opened.
0x34	4	File Attributes	The attributes for the file or directory associated with this record, as returned by the GetFileAttributes function. Attributes of streams associated with the file or directory are excluded.
0x3A	2	FileNameOffset	The offset of the FileName member from the beginning of the structure.
0x3C	N	Filename	The name of the file or directory associated with this record in Unicode format.

REASON FLAG EXAMPLES

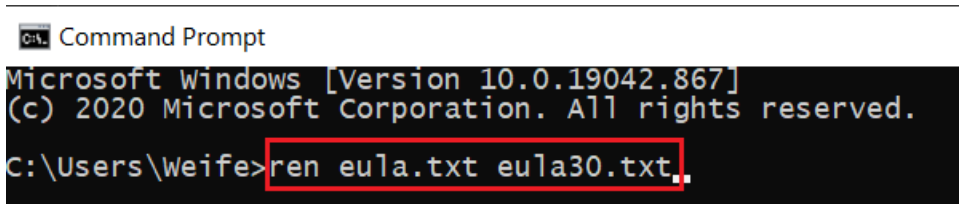
- When a file or directory closes,
  - A final USN record is generated with the **USN_REASON_CLOSE** flag set.
  - The next change (for example, after the next open operation or deletion) starts a new record with a new set of reason flags.
- A rename or move operation
  - Generates two USN records, one that records the old parent directory for the item, and one that records a new parent.

Flag	Value	Description
0x00000100	FILE_CREATE	The file or directory is created for the first time.
0x00000002	DATA_EXTEND	The file or directory is extended (added to).
0x00002000	RENAME_NEW_NAME	A file or directory is renamed, and the file name in the USN_RECORD_V2 structure is the new name.
0x00001000	RENAME_OLD_NAME	The file or directory is renamed, and the file name in the USN_RECORD_V2 structure is the previous name.
0x00000200	FILE_DELETE	The file or directory is deleted.
0x80000000	CLOSE	The file or directory is closed.
0x00008000	BASIC_INFO_CHANGE	A user has either changed one or more file or directory attributes (for example, the read-only, hidden, system, archive, or sparse attribute), or one or more time stamps.
0x00010000	HARD_LINK_CHANGE	An NTFS file system hard link is added to or removed from the file or directory.
0x00000004	DATA_TRUNCATION	The file or directory is truncated.

Hard link: you need a file stored in two different folders.

23. IDENTIFY ALL TRACES RELATED TO ‘RENAMING’ OF FILES IN WINDOWS DESKTOP.

USN JOURNAL WHEN RENAMING A FILE



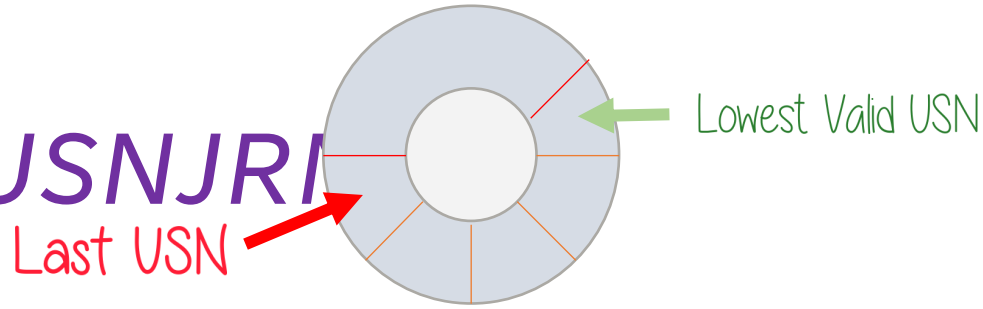
USN Journal

```
eula_30.txt: Rename_New_Name  FileRef: 44657/3  ParentRef: 44361/32
eula_30.txt: Rename_New_Name,Close  FileRef: 44657/3  ParentRef:
44361/32
Eula.txt: Rename_Old_Name  FileRef: 44657/3  ParentRef: 44361/32
```

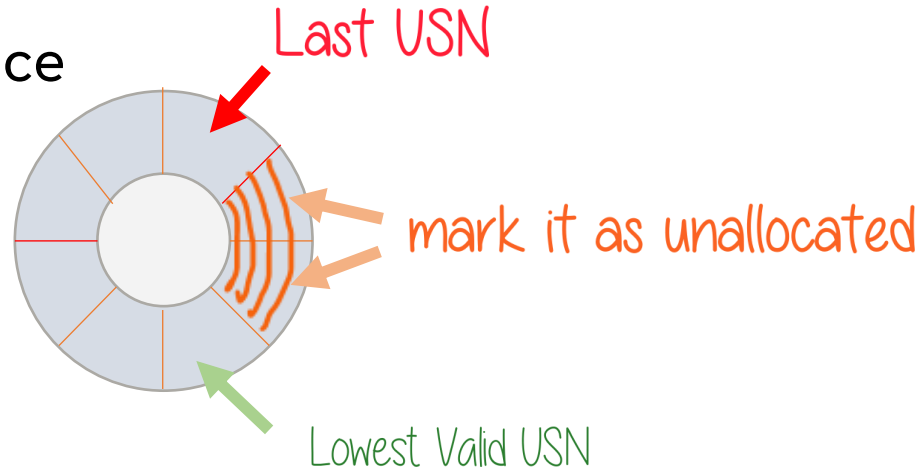
name change

both files have the same reference numbers

HOW TO READ *$EXTEND\$USNJRNL*



- (Scenario 1) If the change journal doesn't reach its maximum size
  - *icat :* extract binary or *https://github.com/jschicht/ExtractUsnJrnl*
  - *bless/xxd* or *UsnJrnl2Csv:* convert to csv
- (Scenario 2) As the change journal reaches its maximum size
  - Clusters of the journal's disk space are marked unallocated by OS
  - That space to be used when needed at a later time
  - *usncarve:* usncarve records in unallocated space
  - *Usnparser:* convert to csv



SCENARIO 1

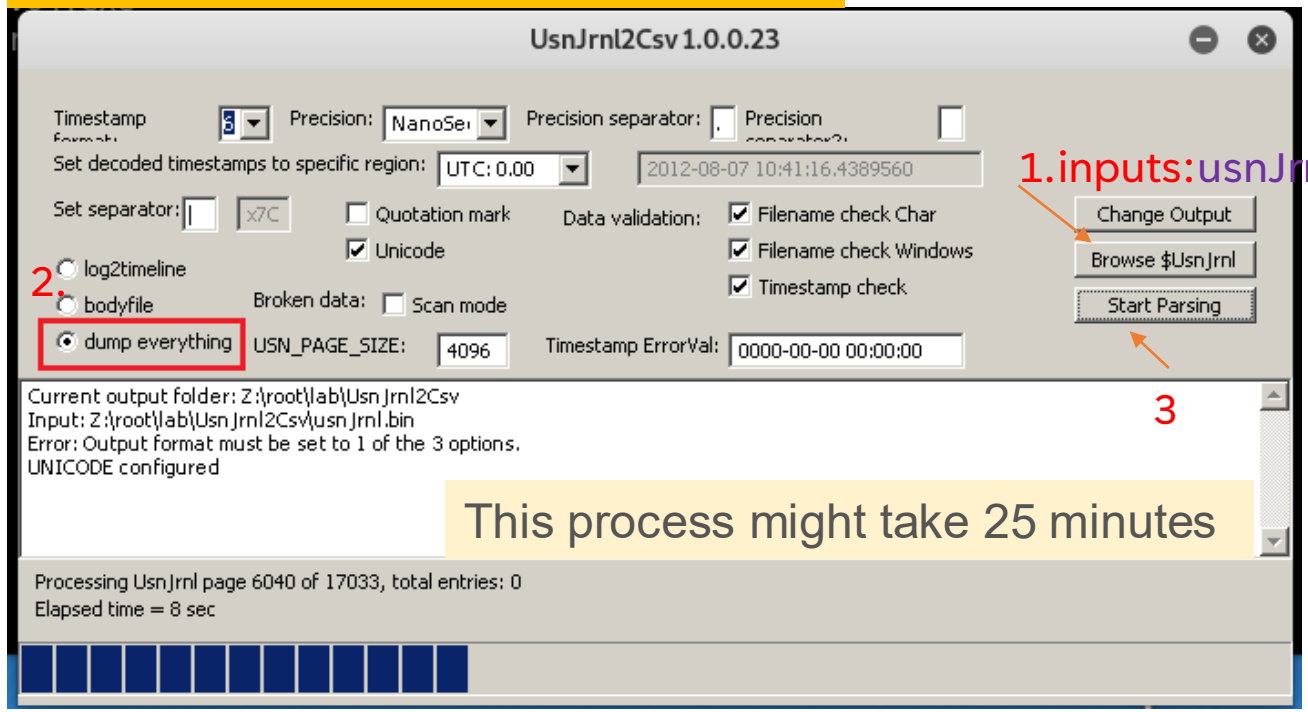
Install *UsnJrnl2Csv* tool

```
/bin/bash 110x32
root@kali:~/lab#
root@kali:~/lab# git clone https://github.com/jschicht/UsnJrnl2Csv.git
Cloning into 'UsnJrnl2Csv'...
remote: Enumerating objects: 176, done.
remote: Total 176 (delta 0), reused 0 (delta 0), pack-reused 176
Receiving objects: 100% (176/176), 12.94 MiB | 4.93 MiB/s, done.
Resolving deltas: 100% (111/111), done.
root@kali:~/lab#
root@kali:~/lab# icat -i raw -o 206848 cfreds_2015_data_leakage_pc.dd 59016-128-3 > UsnJrnl2Csv/usnJrnl.bin
root@kali:~/lab#
root@kali:~/lab# ls UsnJrnl2Csv/usnJrnl.bin -l
-rw-r--r-- 1 root root 69767168 Nov 28 11:47 UsnJrnl2Csv/usnJrnl.bin
root@kali:~/lab#
```

Run *UsnJrnl2.exe* to covert *UsnJrnl.bin* to .csv

```
/bin/bash 103x20
root@kali:~/lab#
root@kali:~/lab# cd UsnJrnl2Csv/
root@kali:~/lab/UsnJrnl2Csv#
root@kali:~/lab/UsnJrnl2Csv# wine UsnJrnl2Csv64.exe
```

Set configuration: Browser to *usnJrnl.bin*



Issue of using *usnJrnl.bin* extracted from *icat*

- *$J may be sparse, which would mean parts of the data is just 00's.*
- *This may be a significant portion of the total data, and most tools will extract this data stream to its full size (which is annoying and a huge waste of disk space).*
- *Consider using the tool https://github.com/jschicht/ExtractUsnJrnl*

https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/NIST_Data_Leakage_Case/NIST_Answers/lab_generated_file/UsnJrnl_2021-10-12_12-36-58.zip

Show the extracted USN journal file

/bin/bash 103x20

```
root@kali:~/lab/UsnJrnl2Csv#
root@kali:~/lab/UsnJrnl2Csv# ls Usn* -l
-rw-r--r-- 1 root root 46649994 Nov 28 13:01 UsnJrnl_2020-11-28_12-15-14.csv
-rw-r--r-- 1 root root      491 Nov 28 13:01 UsnJrnl_2020-11-28_12-15-14.log
-rw-r--r-- 1 root root      915 Nov 28 12:15 UsnJrnl_2020-11-28_12-15-14.sql
-rw-r--r-- 1 root root  1021440 Nov 28 11:45 UsnJrnl2Csv64.exe
-rw-r--r-- 1 root root    63003 Nov 28 11:45 UsnJrnl2Csv.au3
-rw-r--r-- 1 root root   902656 Nov 28 11:45 UsnJrnl2Csv.exe
root@kali:~/lab/UsnJrnl2Csv#
```

/bin/bash 101x35

```
root@kali:~/lab/UsnJrnl2Csv#
root@kali:~/lab/UsnJrnl2Csv# cat UsnJrnl_2020-11-28_12-15-14.csv | head -n 2
Offset|FileName|USN|Timestamp|Reason|MFTReference|MFTReferenceSeqNo|MFTParentReference|MFTParentRefer
enceSeqNo|FileAttributes|MajorVersion|MinorVersion|SourceInfo|SecurityId
0x01F80000|api-ms-win-core-fibers-l1-1-0.dll|33030144|2015-03-22 15:15:08.2192456|BASIC_INFO_CHANGE+C
LOSE+INDEXABLE_CHANGE|63536|4|63453|4|archive|2|0|0x00000000|0
root@kali:~/lab/UsnJrnl2Csv#
```

Value	Description
0x01	A file that is read-only.
0x02	The file or directory is hidden
0x04	A file or directory that the operating system uses a part of, or uses exclusively.
0x10	The handle that identifies a directory.
0x20	An archive file or directory.
0x40	This value is reserved for system use
0x80	A file that does not have other attributes set.
0x100	A file that is being used for temporary storage.
0x200	A file that is a sparse file.
0x400	A file or directory that has an associated reparse point, or a file that is a symbolic link.
0x800	A file or directory that is compressed
0x1000	This attribute indicates that the file data is physically moved to offline storage.
0x2000	The file or directory is not to be indexed by the content indexing service
0x4000	A file or directory that is encrypted.
0x8000	The directory or user data stream is configured with integrity (only supported on ReFS volumes).
0x10000	This value is reserved for system use
0x20000	The user data stream not to be read by the background data integrity scanner (AKA scrubber).

the name is confusing: it is MFT entry ID/inode

Offset|FileName|USN|Timestamp|Reason|MFTReference|MFTReferenceSeqNo|MFTParentReference|MFTParentReferenceSeqNo|FileAttributes|MajorVersion|MinorVersion|SourceInfo|SecurityId

Only show "*RENAME_OLD*" information of any .jpg files

/bin/bash 141x20

```
root@kali:~/lab/UsnJrnl2Csv#
root@kali:~/lab/UsnJrnl2Csv# grep -P "\.jpg.*RENAME_OLD"  UsnJrnl_2020-11-28_12-15-14.csv --color
0x03D6BCA0|Chrysanthemum.jpg|64404640|2015-03-24 20:11:42.8361742|RENAME_OLD_NAME|74620|285|21921|3|archive|2|0|0x00000000|0
0x03D6BF68|Desert.jpg|64405352|2015-03-24 20:11:42.8517742|RENAME_OLD_NAME|74625|10|21921|3|archive|2|0|0x00000000|0
0x03D6C528|Hydrangeas.jpg|64406824|2015-03-24 20:11:42.8517742|RENAME_OLD_NAME|74641|10|21921|3|archive|2|0|0x00000000|0
0x03D6CAC8|Jellyfish.jpg|64408264|2015-03-24 20:11:42.8517742|RENAME_OLD_NAME|74656|8|21921|3|archive|2|0|0x00000000|0
0x03D6CD88|Koala.jpg|64408968|2015-03-24 20:11:42.8517742|RENAME_OLD_NAME|74666|10|21921|3|archive|2|0|0x00000000|0
0x03D6D058|Lighthouse.jpg|64409688|2015-03-24 20:11:42.8517742|RENAME_OLD_NAME|74667|16|21921|3|archive|2|0|0x00000000|0
0x03D6D318|Penguins.jpg|64410392|2015-03-24 20:11:42.8517742|RENAME_OLD_NAME|74670|12|21921|3|archive|2|0|0x00000000|0
0x03D6D5D8|Tulips.jpg|64411096|2015-03-24 20:11:42.8517742|RENAME_OLD_NAME|74676|10|21921|3|archive|2|0|0x00000000|0
0x0411B558|Chrysanthemum.jpg|68269400|2015-03-25 15:13:39.1698055|RENAME_OLD_NAME|74620|285|74402|9|not_indexed|2|0|0x00000000|0
0x0411C4F8|Desert.jpg|68273400|2015-03-25 15:13:39.4350055|RENAME_OLD_NAME|74625|10|74402|9|not_indexed|2|0|0x00000000|0
0x0411D268|Hydrangeas.jpg|68276840|2015-03-25 15:13:39.5442055|RENAME_OLD_NAME|74641|10|74402|9|not_indexed|2|0|0x00000000|0
0x0411F570|Jellyfish.jpg|68285808|2015-03-25 15:13:48.8730055|RENAME_OLD_NAME|74656|8|74402|9|not_indexed|2|0|0x00000000|0
0x04120420|Koala.jpg|68289568|2015-03-25 15:13:49.0602055|RENAME_OLD_NAME|74666|10|74402|9|not_indexed|2|0|0x00000000|0
0x041211B8|Lighthouse.jpg|68293048|2015-03-25 15:13:49.2162055|RENAME_OLD_NAME|74667|16|74402|9|not_indexed|2|0|0x00000000|0
0x04122058|Penguins.jpg|68296792|2015-03-25 15:13:49.3566055|RENAME_OLD_NAME|74670|12|74402|9|not_indexed|2|0|0x00000000|0
0x04122EC8|Tulips.jpg|68300488|2015-03-25 15:13:49.4658055|RENAME_OLD_NAME|74676|10|74402|9|not_indexed|2|0|0x00000000|0
root@kali:~/lab/UsnJrnl2Csv#
```

- Timestamps are written UTC 0.00 by default. You have to configure it.
- With NTFS journal file only, it may be hard to find full paths.
- (+ $MFT for identifying full paths of files)
- You can consider the Registry *ShellBags* for further information.

Offset|FileName|USN|Timestamp|Reason|MFTReference|MFTReferenceSeqNo|MFTParentReference|MFTParentReferenceSeqNo|FileAttributes|MajorVersion|MinorVersion|SourceInfo|SecurityId

Only show "*RENAME_OLD*" information of any *.jpg* files. Only show three columns, FileName, Timestamp, Reason

```
/bin/bash 97x35
root@kali:~/lab/UsnJrnl2Csv#
root@kali:~/lab/UsnJrnl2Csv# awk '{print NR, $2, $4, $5}' FS='|' UsnJrnl_2020-11-28_12-15-14.csv
 | grep -E "\.jpg.*RENAME_OLD" --color
261131 Chrysanthemum.jpg 2015-03-24 20:11:42.8361742 RENAME_OLD_NAME
261139 Desert.jpg 2015-03-24 20:11:42.8517742 RENAME_OLD_NAME
261155 Hydrangeas.jpg 2015-03-24 20:11:42.8517742 RENAME_OLD_NAME
261171 Jellyfish.jpg 2015-03-24 20:11:42.8517742 RENAME_OLD_NAME
261179 Koala.jpg 2015-03-24 20:11:42.8517742 RENAME_OLD_NAME
261187 Lighthouse.jpg 2015-03-24 20:11:42.8517742 RENAME_OLD_NAME
261195 Penguins.jpg 2015-03-24 20:11:42.8517742 RENAME_OLD_NAME
261203 Tulips.jpg 2015-03-24 20:11:42.8517742 RENAME_OLD_NAME
299494 Chrysanthemum.jpg 2015-03-25 15:13:39.1698055 RENAME_OLD_NAME
299536 Desert.jpg 2015-03-25 15:13:39.4350055 RENAME_OLD_NAME
299578 Hydrangeas.jpg 2015-03-25 15:13:39.5442055 RENAME_OLD_NAME
299665 Jellyfish.jpg 2015-03-25 15:13:48.8730055 RENAME_OLD_NAME
299707 Koala.jpg 2015-03-25 15:13:49.0602055 RENAME_OLD_NAME
299749 Lighthouse.jpg 2015-03-25 15:13:49.2162055 RENAME_OLD_NAME
299791 Penguins.jpg 2015-03-25 15:13:49.3566055 RENAME_OLD_NAME
299833 Tulips.jpg 2015-03-25 15:13:49.4658055 RENAME_OLD_NAME
root@kali:~/lab/UsnJrnl2Csv#
```

How to find the new file name?

Show what has happened around a specific time with the same MFT reference #

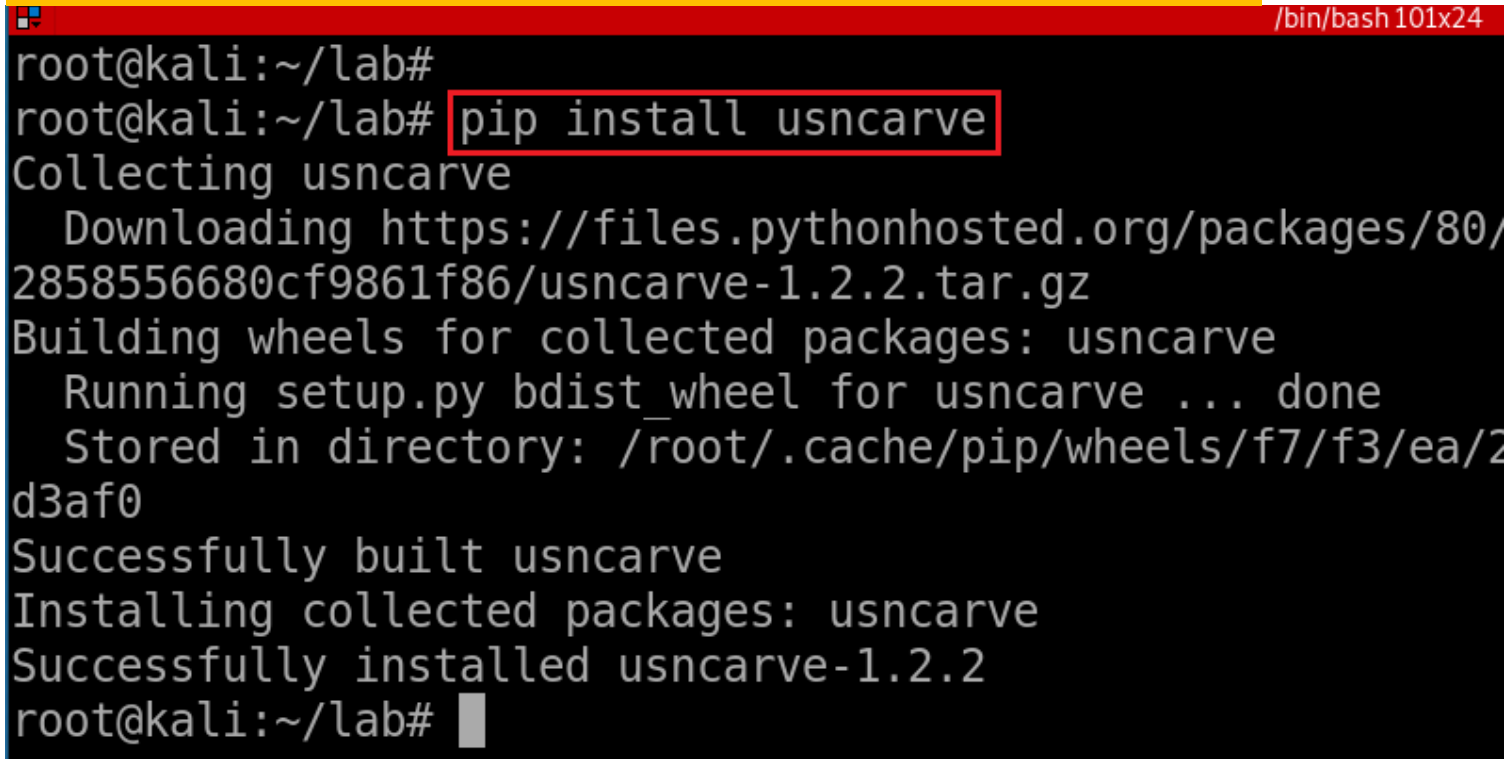
```
└─# grep  "2015-03-24 20:11.*|74620|"  UsnJrnl_2021-10-12_12-36-58.csv
0x03D6BCA0|Chrysanthemum.jpg|64404640|2015-03-24 20:11:42.8361742|RENAME_OLD_NAME|74620|285|21921|3|archiv
e|2|0|0x00000000|0
0x03D6BD00|$RKXD1U3.jpg|64404736|2015-03-24 20:11:42.8361742|RENAME_NEW_NAME|74620|285|15721|2|archive|2|0
|0x00000000|0
0x03D6BD58|$RKXD1U3.jpg|64404824|2015-03-24 20:11:42.8361742|CLOSE+RENAME_NEW_NAME|74620|285|15721|2|archi
ve|2|0|0x00000000|0
0x03D6BDB0|$RKXD1U3.jpg|64404912|2015-03-24 20:11:42.8361742|SECURITY_CHANGE|74620|285|15721|2|archive|2|0
|0x00000000|0
0x03D6BE08|$RKXD1U3.jpg|64405000|2015-03-24 20:11:42.8361742|CLOSE+SECURITY_CHANGE|74620|285|15721|2|archi
ve|2|0|0x00000000|0
```

NOTES: USN AND MALWARE INFECTION

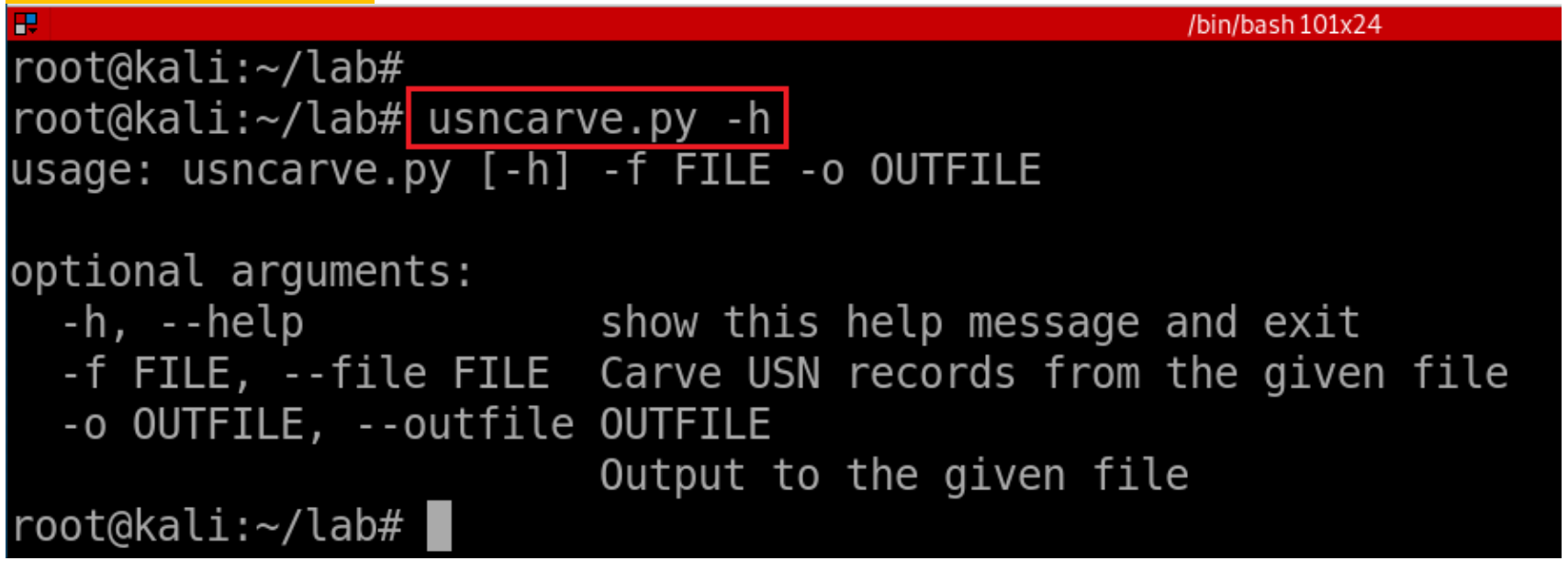
- The use of the USN change journal can also be useful in identifying activity that occurs during a malware infection. For example, in some cases, malware may create a downloader, use that to download another bit of malware, and then delete the original downloader. The USN change journal can help you identify that activity, even if the MFT record for the original downloader has been reused and overwritten.

SCENARIO 2

Install *usncarve* tool (note we have changed the path)

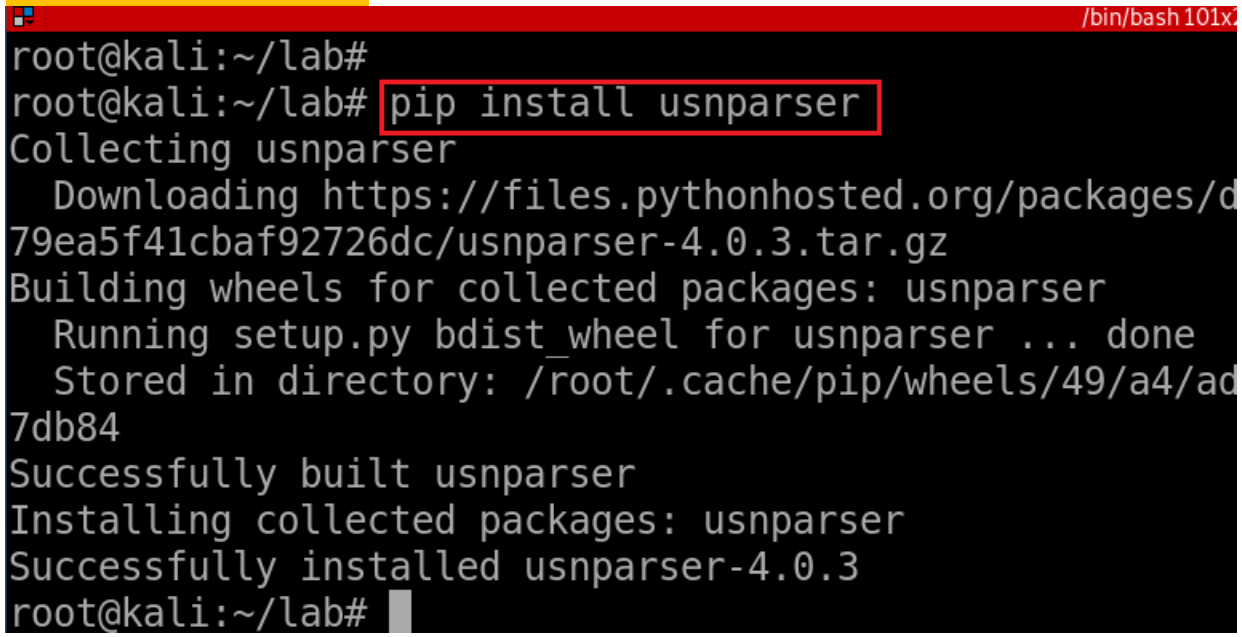


Verify

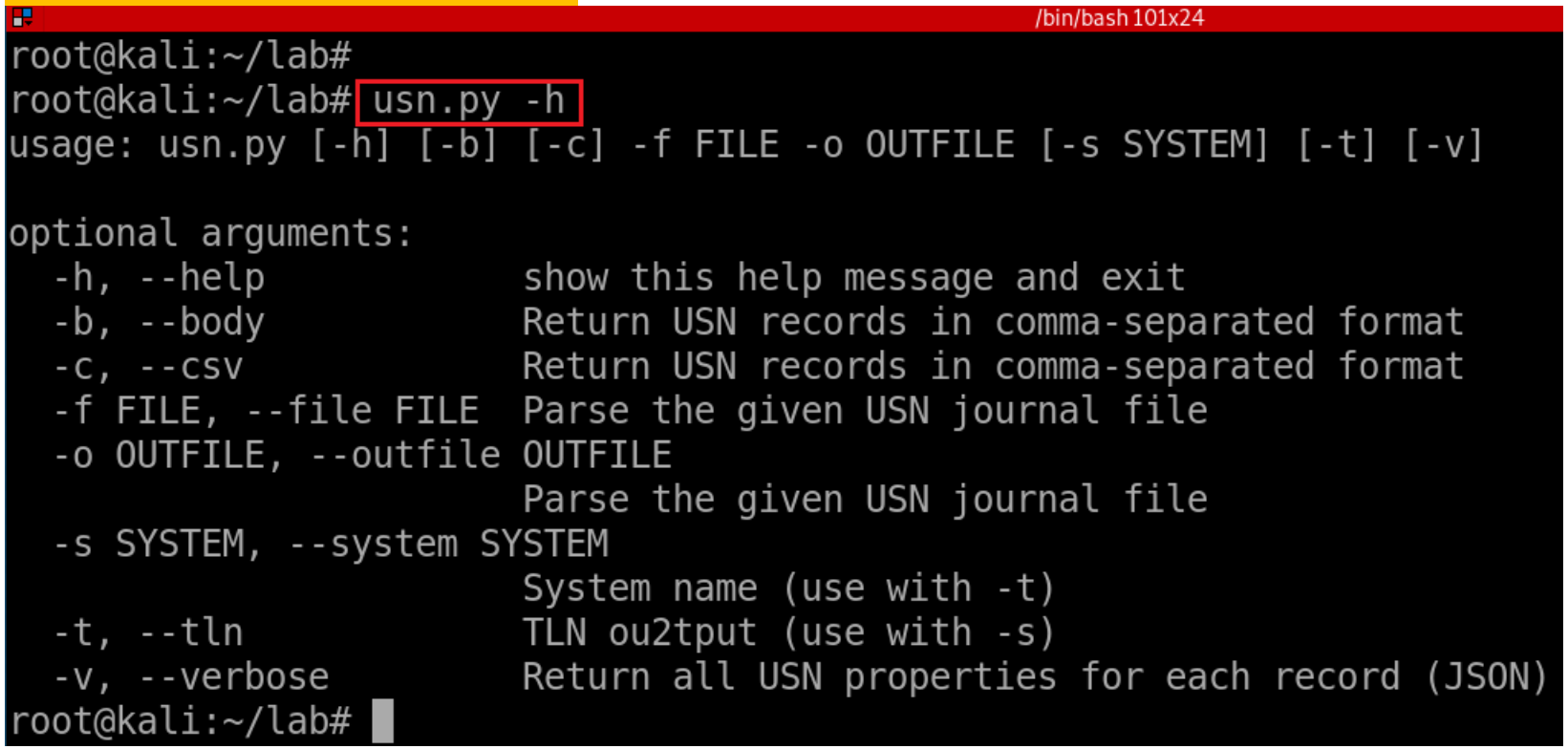

/bin/bash 101x24
root@kali:~/lab#
root@kali:~/lab# usncarve.py -h
usage: usncarve.py [-h] -f FILE -o OUTFILE
optional arguments:
-h, --help show this help message and exit
-f FILE, --file FILE Carve USN records from the given file
-o OUTFILE, --outfile OUTFILE
Output to the given file
root@kali:~/lab#

INSTALL *USNPARSER* TOOL

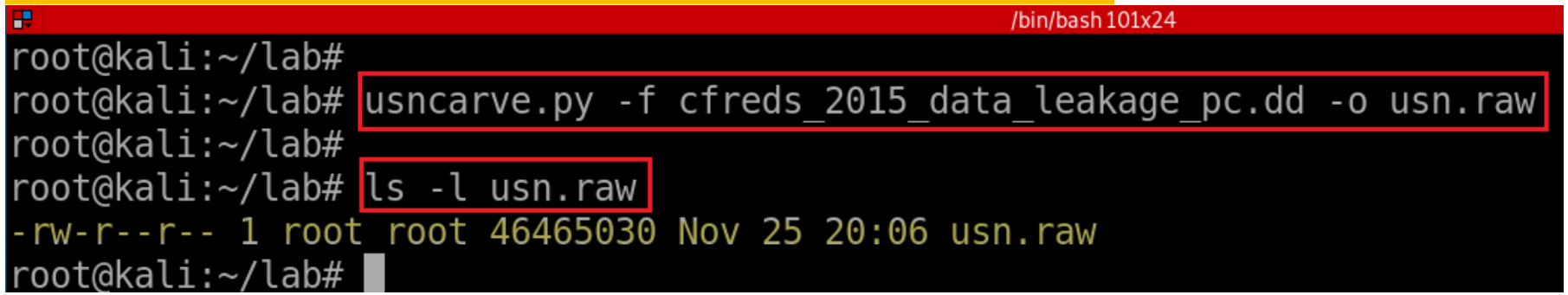
Install via pip



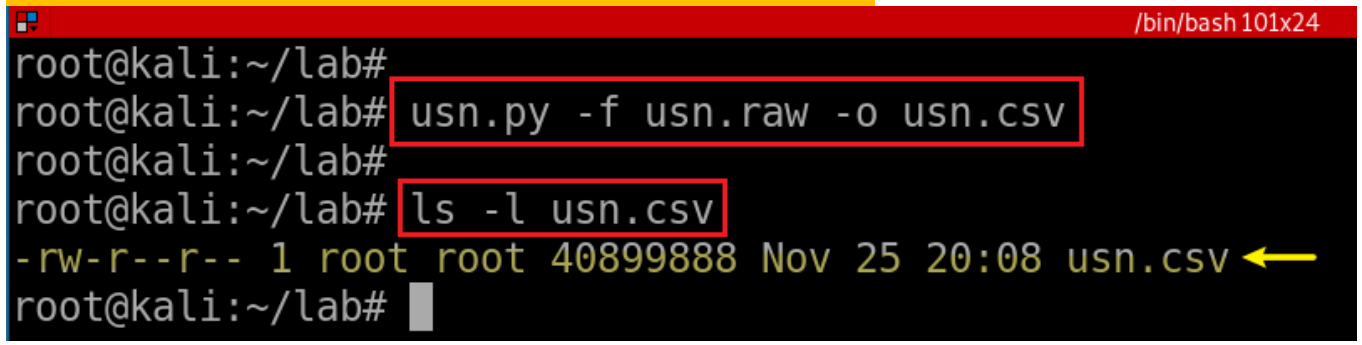
Verify installation



usncarve.py *-f* input file *-o* output file



usn.py *-f* input file *-o* output file

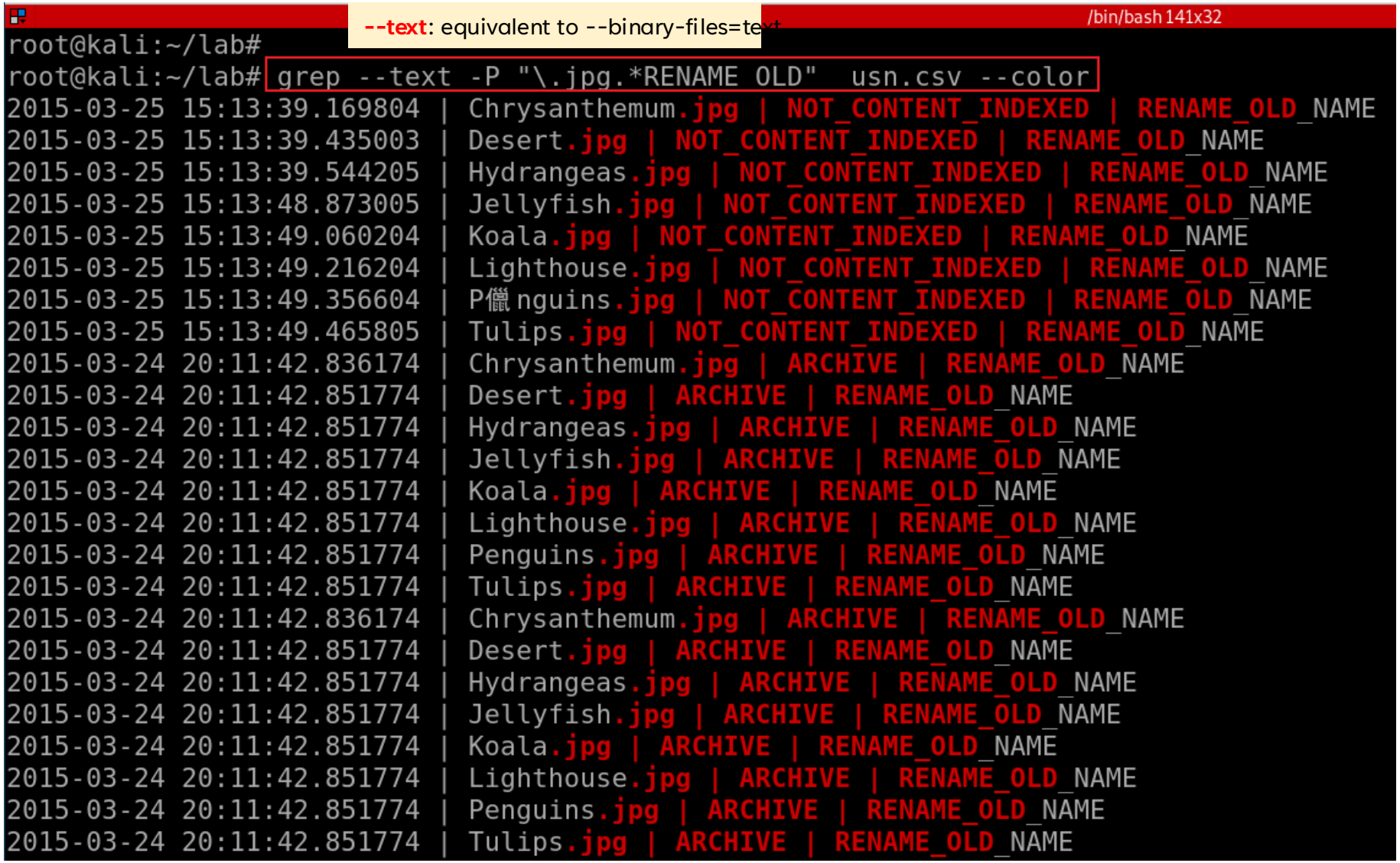


View renamed files

```
/bin/bash 120x24
root@kali:~/lab#
root@kali:~/lab# cat usn.csv | grep -i rename | head
2015-03-22 15:14:17.097954 | edb0043D.log | ARCHIVE NOT_CONTENT_INDEXED | RENAME_OLD_NAME
2015-03-22 15:14:17.097954 | edbtmp.log | ARCHIVE NOT_CONTENT_INDEXED | RENAME_NEW_NAME
2015-03-22 15:14:17.097954 | edbtmp.log | ARCHIVE NOT_CONTENT_INDEXED | RENAME_NEW_NAME CLOSE
2015-03-22 15:14:17.097954 | edb.log | ARCHIVE NOT_CONTENT_INDEXED | RENAME_OLD_NAME
2015-03-22 15:14:17.097954 | edb0044E.log | ARCHIVE NOT_CONTENT_INDEXED | RENAME_NEW_NAME
2015-03-22 15:14:17.097954 | edb0044E.log | ARCHIVE NOT_CONTENT_INDEXED | RENAME_NEW_NAME CLOSE
2015-03-22 15:14:17.113554 | edbtmp.log | ARCHIVE NOT_CONTENT_INDEXED | RENAME_OLD_NAME
2015-03-22 15:14:17.113554 | edb.log | ARCHIVE NOT_CONTENT_INDEXED | RENAME_NEW_NAME
2015-03-22 15:14:17.113554 | edb.log | ARCHIVE NOT_CONTENT_INDEXED | RENAME_NEW_NAME CLOSE
2015-03-22 15:14:18.174358 | edb.log | ARCHIVE NOT_CONTENT_INDEXED | RENAME_OLD_NAME
root@kali:~/lab#
root@kali:~/lab# cat usn.csv | grep -i rename | wc -l
46 ←
root@kali:~/lab#
```

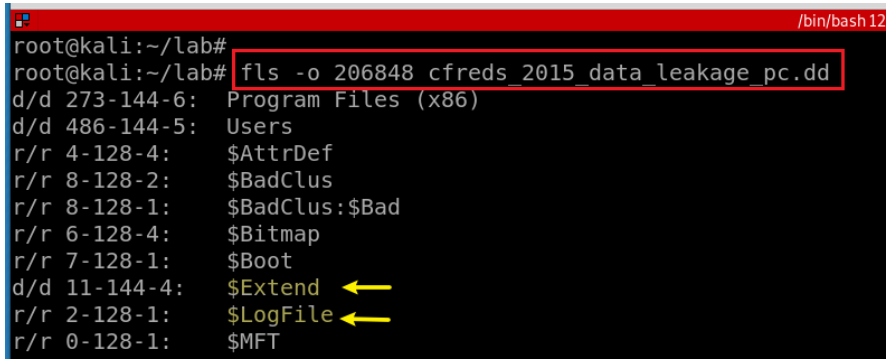
- NTFS journal file analysis (→ $UsnJrnl)
- \$Extend\$UsnJrnl·$J (+ $MFT for identifying full paths of files)

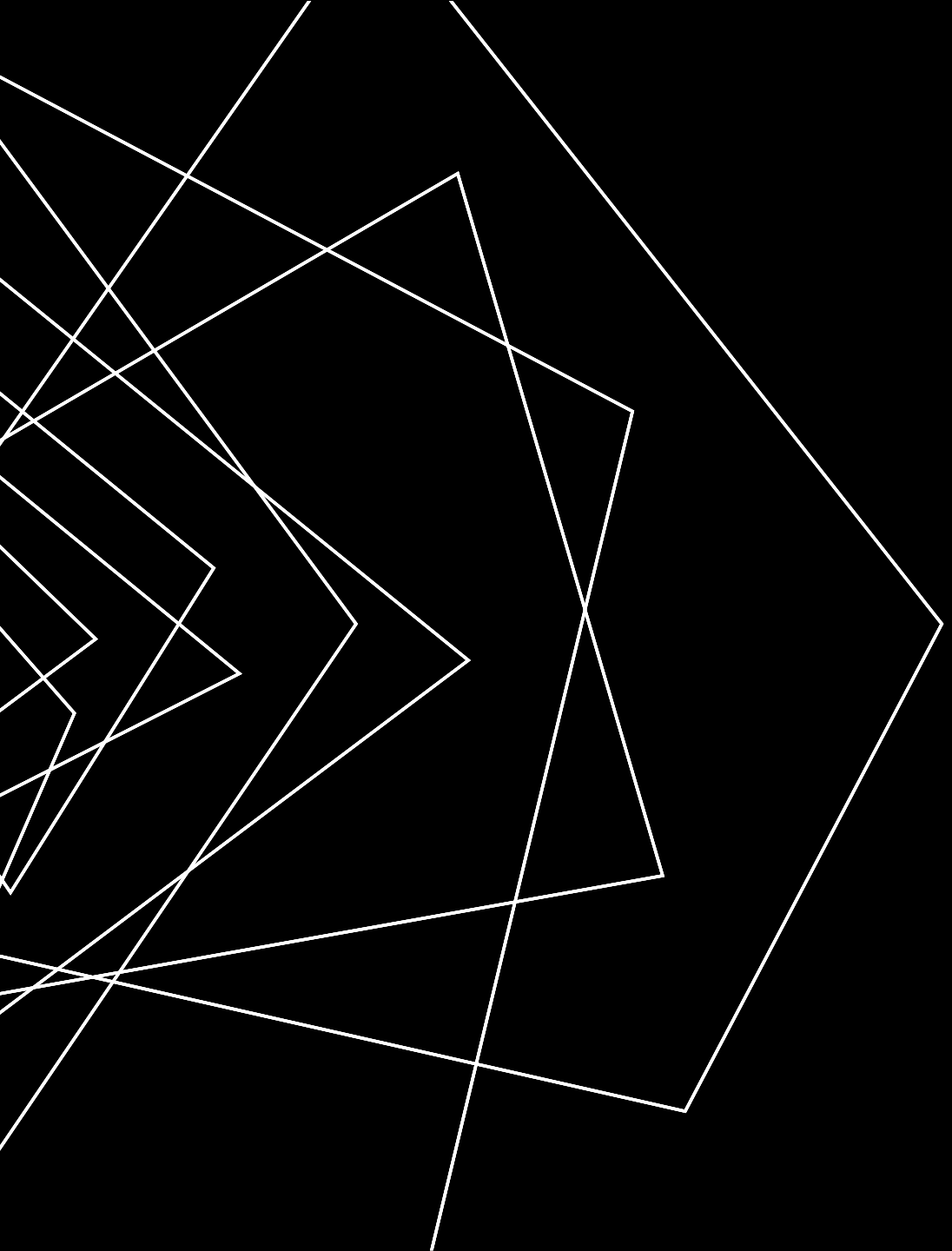
- You can also consider the Windows Search database. (See Questions 46)


--text: equivalent to --binary-files=text
/bin/bash 141x32
root@kali:~/lab#
root@kali:~/lab# grep --text -P "\.jpg.*RENAME_OLD" usn.csv --color
2015-03-25 15:13:39.169804 | Chrysanthemum.jpg | NOT_CONTENT_INDEXED | RENAME_OLD_NAME
2015-03-25 15:13:39.435003 | Desert.jpg | NOT_CONTENT_INDEXED | RENAME_OLD_NAME
2015-03-25 15:13:39.544205 | Hydrangeas.jpg | NOT_CONTENT_INDEXED | RENAME_OLD_NAME
2015-03-25 15:13:48.873005 | Jellyfish.jpg | NOT_CONTENT_INDEXED | RENAME_OLD_NAME
2015-03-25 15:13:49.060204 | Koala.jpg | NOT_CONTENT_INDEXED | RENAME_OLD_NAME
2015-03-25 15:13:49.216204 | Lighthouse.jpg | NOT_CONTENT_INDEXED | RENAME_OLD_NAME
2015-03-25 15:13:49.356604 | P儷nguins.jpg | NOT_CONTENT_INDEXED | RENAME_OLD_NAME
2015-03-25 15:13:49.465805 | Tulips.jpg | NOT_CONTENT_INDEXED | RENAME_OLD_NAME
2015-03-24 20:11:42.836174 | Chrysanthemum.jpg | ARCHIVE | RENAME_OLD_NAME
2015-03-24 20:11:42.851774 | Desert.jpg | ARCHIVE | RENAME_OLD_NAME
2015-03-24 20:11:42.851774 | Hydrangeas.jpg | ARCHIVE | RENAME_OLD_NAME
2015-03-24 20:11:42.851774 | Jellyfish.jpg | ARCHIVE | RENAME_OLD_NAME
2015-03-24 20:11:42.851774 | Koala.jpg | ARCHIVE | RENAME_OLD_NAME
2015-03-24 20:11:42.851774 | Lighthouse.jpg | ARCHIVE | RENAME_OLD_NAME
2015-03-24 20:11:42.851774 | Penguins.jpg | ARCHIVE | RENAME_OLD_NAME
2015-03-24 20:11:42.851774 | Tulips.jpg | ARCHIVE | RENAME_OLD_NAME
2015-03-24 20:11:42.836174 | Chrysanthemum.jpg | ARCHIVE | RENAME_OLD_NAME
2015-03-24 20:11:42.851774 | Desert.jpg | ARCHIVE | RENAME_OLD_NAME
2015-03-24 20:11:42.851774 | Hydrangeas.jpg | ARCHIVE | RENAME_OLD_NAME
2015-03-24 20:11:42.851774 | Jellyfish.jpg | ARCHIVE | RENAME_OLD_NAME
2015-03-24 20:11:42.851774 | Koala.jpg | ARCHIVE | RENAME_OLD_NAME
2015-03-24 20:11:42.851774 | Lighthouse.jpg | ARCHIVE | RENAME_OLD_NAME
2015-03-24 20:11:42.851774 | Penguins.jpg | ARCHIVE | RENAME_OLD_NAME
2015-03-24 20:11:42.851774 | Tulips.jpg | ARCHIVE | RENAME_OLD_NAME

USN JOURNAL VS. NTFS FILE SYSTEM JOURNALING

- NTFS file journaling uses the NTFS Log *($LogFile)* to record metadata changes to the volume.
  - USN Journal only has change flags, no metadata.
- NTFS file journaling ensures that its complex internal data structures will remain consistent in case of
  - system crashes or data moves performed by the defragmentation API,
- NTFS file journaling allows easy rollback of uncommitted changes to these critical data structures when the volume is remounted.





THANK YOU

Yau Ka Cheung
Yauka.Cheung@etamu.edu
ACB1 Room 335